# PROJECT REPORT

## Healthcare Data Analysis

Using Python, MySQL and PostgreSQL

**Submitted By:**

Aryan Nagori

GitHub: github.com/AryanNagori1405

**Technologies Used:**

Python | pandas | MySQL | PostgreSQL | SQL | pytest | pytest-cov

Date: October 2026

# TABLE OF CONTENTS

1. Introduction
2. Problem Statement
3. Objectives
4. Dataset Description
5. Data Cleaning and Preprocessing
6. Project Structure and File Organization
7. Python Implementation: 6 Functions
8. SQL Implementation: 10 Queries
9. Database Creation: MySQL and PostgreSQL
10. Testing and Code Coverage
11. Key Findings with Real Numbers
12. How to Run the Project (Step by Step)
13. Requirements and Dependencies
14. Conclusion

## 1. INTRODUCTION

This project presents a complete end-to-end healthcare data analysis. It takes a real-world hospital admissions dataset from Kaggle, cleans it with Python and pandas, stores it in two different relational databases (MySQL and PostgreSQL), and answers real analytical questions using SQL queries. The code is broken into reusable Python functions, thoroughly unit-tested with pytest, and documented for anyone who wants to understand or extend it.

The workflow follows the classic data analysis pipeline: (1) load and explore raw data, (2) clean and fix quality issues, (3) transform and enrich the data, (4) load it into databases, (5) analyze with SQL, and (6) test and document everything.

## 2. PROBLEM STATEMENT

Healthcare organizations generate large volumes of patient data across admissions, billing, diagnoses, medications and test results. Answering questions like "how many admissions happen each month?", "which diagnosis is most common?", or "which hospital earns the most?" requires clean data and well-written queries. The goal of this project is to demonstrate this entire process on a real dataset of over 55,000 patient records.

## 3. OBJECTIVES

1. Load and explore a public healthcare dataset from Kaggle (55,500 rows, 15 columns).
2. Identify and fix data quality problems: duplicates, inconsistent text, wrong data types.
3. Clean and transform the data using Python and pandas.
4. Create the same table structure in both MySQL and PostgreSQL and import the cleaned data.
5. Write 10 SQL queries that answer real business questions, including joins, GROUP BY, and at least two window functions (RANK and running total).
6. Convert the cleaning and loading steps into 6 reusable Python functions in a .py file.
7. Write 12 pytest tests for those functions and measure coverage with pytest-cov (target: 90% or more).
8. Document the entire project: README, cleaning report, and this PDF.

## 4. DATASET DESCRIPTION

Source: Kaggle Healthcare Dataset (hospital admissions / patient billing data).

Original Shape: 55,500 rows and 15 columns.

Cleaned Shape: 54,966 rows and 15 columns (after removing 534 duplicate rows).

Missing Values: 0 (the dataset had no nulls).

### Columns:

```
Name, Age, Gender, Blood Type, Medical Condition, Date of Admission,
Doctor, Hospital, Insurance Provider, Billing Amount, Room Number,
Admission Type, Discharge Date, Medication, Test Results
```

### Data Types After Cleaning:

```
date_of_admission : datetime64
discharge_date    : datetime64
billing_amount    : float64
age, room_number  : integer
text columns      : string (standardized to Title Case)
duration_days     : integer (derived column)
```

## 5. DATA CLEANING AND PREPROCESSING

The raw dataset contained several quality issues that had to be fixed before analysis:

### Issues Found:

1. Duplicate rows: 534 exact duplicate records were found.

2. Inconsistent text casing: Values like "Bobby JacksOn", "andrEw waTtS", "MALE"/"male" mixed across columns.

3. Dates stored as strings in "dd-mm-yyyy" format instead of proper date type.

4. Billing Amount needed to be guaranteed numeric.

### Fixes Applied:

1. Duplicates removed: 534 rows deleted. Standardization ran BEFORE deduplication so that "male" and "MALE" became identical rows and were then correctly removed.

2. Text standardized: strip() and title() applied to Name, Gender, Blood Type, Medical Condition, Doctor, Hospital, Insurance Provider, Admission Type, Medication, Test Results. All rows affected (formatting only).

3. Dates converted to datetime using format %d-%m-%Y for Date of Admission and Discharge Date.

4. Billing Amount cast to numeric.

5. New feature Duration_Days = Discharge Date - Admission Date, useful for length-of-stay analysis.

All of this is documented in cleaning_analysis.md with affected row counts.

## 6. PROJECT STRUCTURE AND FILE ORGANIZATION

```
HealthcareDataAnalysis/
|-- data/
|   |-- healthcare_cleaned.csv      (cleaned data)
|   |-- healthcare_for_db.csv        (data + Duration_Days for DB import)
|-- notebooks/
|   |-- data_exploration.ipynb       (exploratory analysis notebook)
|-- src/
|   |-- data_processing.py           (6 reusable Python functions)
|-- sql/
|   |-- create_tables.sql            (PostgreSQL table schema)
|   |-- create_tables_mysql.sql      (MySQL table schema)
|   |-- queries.sql                  (10 analytical queries)
|-- tests/
|   |-- __init__.py
|   |-- test_data_processing.py      (12 pytest tests)
|-- cleaning_analysis.md             (data cleaning documentation)
|-- README.md                        (project overview and run guide)
|-- db_import.py                     (database import helper)
|-- PROJECT_REPORT.pdf               (this document)
```

Each folder has a clear responsibility: data/ holds CSV files, src/ holds reusable logic, sql/ holds all database scripts, tests/ holds the test suite, and notebooks/ holds exploration work.

## 7. PYTHON IMPLEMENTATION: 6 FUNCTIONS

All cleaning and loading logic lives in src/data_processing.py as six well-documented functions:

### 1. load_data(filepath)

Reads a CSV file into a pandas DataFrame. Raises FileNotFoundError with a clear message if the file does not exist.

### 2. check_missing_values(df)

Returns a pandas Series with the count of missing (NaN) values in each column. Used during exploration to confirm the dataset had zero missing values.

### 3. check_duplicates(df)

Returns the number of duplicate rows in the DataFrame. Reported 534 duplicates in the raw dataset.

### 4. clean_data(df)

The core cleaning function. Order matters: it first standardizes text (strip + title case), then converts dates and billing to correct types, and only THEN removes duplicates. This order ensures rows that only differed in casing are correctly identified as duplicates. Returns the cleaned DataFrame.

### 5. save_cleaned_data(df, output_path)

Saves the DataFrame to CSV. Creates the output directory automatically if it does not exist (useful for nested paths).

### 6. load_and_clean(input_path, output_path=None)

End-to-end convenience function: loads the raw CSV, cleans it, and optionally saves the result. Returns the cleaned DataFrame.

## 8. SQL IMPLEMENTATION: 10 QUERIES

All 10 queries are in sql/queries.sql and run on both MySQL and PostgreSQL. They cover joins, GROUP BY, aggregations and two window functions as required.

### Q1. Admissions per month

Groups by year and month using EXTRACT, counts admissions to see seasonal patterns.

### Q2. Average cost per department

Uses AVG(billing_amount) grouped by medical_condition to compare costs across conditions.

### Q3. Top 5 most common diagnoses

COUNT grouped by medical_condition, ordered descending with LIMIT 5.

### Q4. Total billing by insurance provider

SUM and AVG of billing grouped by insurance_provider to see which insurer pays most.

### Q5. Average length of stay by condition

AVG(duration_days) grouped by medical_condition.

### Q6. Admissions by admission type

COUNT and AVG cost grouped by admission_type (Emergency, Elective, Urgent).

### Q7. Top 10 doctors by patient count

COUNT and SUM revenue grouped by doctor, LIMIT 10.

### Q8. RANK() window function

RANK() OVER (ORDER BY SUM(billing_amount) DESC) to rank hospitals by total revenue.

### Q9. Running total by month

SUM(billing_amount) OVER (ORDER BY year, month) gives a cumulative running total of billing across months.

### Q10. Demographics by gender and test results

AVG age and billing grouped by gender and test_results for demographic insights.

## 9. DATABASE CREATION: MYSQL AND POSTGRESQL

The same table structure was created in both databases and the cleaned data imported into each.

### MySQL:

```
CREATE DATABASE healthcare_db;
USE healthcare_db;
-- then run sql/create_tables_mysql.sql (uses AUTO_INCREMENT)
-- import data/healthcare_for_db.csv via LOAD DATA or Python/SQLAlchemy
```

### PostgreSQL:

```
CREATE DATABASE healthcare_db;
-- then run sql/create_tables.sql (uses SERIAL)
-- import data via \copy or Python/SQLAlchemy
```

### Table Schema (common to both):

```
id, name, age, gender, blood_type, medical_condition,
date_of_admission, doctor, hospital, insurance_provider,
billing_amount, room_number, admission_type, discharge_date,
medication, test_results, duration_days
```

Verification: SELECT COUNT(*) FROM healthcare; should return 54,966 rows on both databases. All 10 queries run successfully on both MySQL and PostgreSQL.

## 10. TESTING AND CODE COVERAGE

The test suite lives in tests/test_data_processing.py and contains 12 pytest tests covering all 6 functions, including edge cases:

- test_load_data_success: loads a small CSV correctly.
- test_load_data_file_not_found: raises FileNotFoundError for missing files.
- test_check_missing_values: verifies null counting per column.
- test_check_duplicates: verifies duplicate row counting.
- test_clean_data_removes_duplicates: confirms duplicates removed only after text standardization.
- test_clean_data_handles_dates: confirms date columns become datetime.
- test_clean_data_handles_billing_numeric: confirms billing becomes numeric.
- test_save_cleaned_data: confirms file is written and readable back.
- test_load_and_clean: confirms end-to-end flow works.
- test_clean_data_preserves_column_structure: no columns lost during cleaning.
- test_clean_data_handles_missing_columns_gracefully: function does not crash on incomplete data.
- test_save_without_makedirs: nested output directories are created automatically.

**Result: 12 tests passed. Coverage: 100% (pytest-cov), well above the 90% target.**

```
$env:PYTHONPATH="C:\Users\aryan\HealthcareDataAnalysis"
& "C:\Program Files\Python314\python.exe" -m pytest tests/
    --cov=src --cov-report=term-missing
```

## 11. KEY FINDINGS WITH REAL NUMBERS

The analysis of 54,966 patient records produced these concrete findings:

### Finding 1: Overall Average Billing

The average billing amount across all 54,966 patients is $25,544.31. This baseline helps identify which conditions and providers are above or below average.

### Finding 2: Top Medical Conditions by Volume

The six conditions are remarkably evenly distributed, indicating a balanced synthetic sample:

Arthritis: 9,218 cases | Diabetes: 9,216 cases | Hypertension: 9,151 cases

Obesity: 9,146 cases | Cancer: 9,140 cases | Asthma: 9,095 cases

### Finding 3: Average Cost Varies by Condition

Obesity is the most expensive condition at $25,804.36 average, while Cancer is the least at $25,152.32:

Obesity $25,804.36 > Diabetes $25,660.48 > Asthma $25,633.46

Arthritis $25,511.78 > Hypertension $25,503.06 > Cancer $25,152.32

### Additional Observations:

- Monthly admissions peak at 899 in June 2019 and range between roughly 4,200 and 4,700 per month across the full dataset years.

- Hospital names are highly fragmented (39,876 unique values for 54,966 rows), suggesting many small providers or name inconsistencies worth further normalization.

## 12. HOW TO RUN THE PROJECT (STEP BY STEP)

### Step 1: Install dependencies

```
pip install pandas mysql-connector-python sqlalchemy psycopg2-binary pytest pytest-cov
```

### Step 2: Set the Python path (Windows PowerShell)

```
$env:PYTHONPATH="C:\Users\aryan\HealthcareDataAnalysis"
```

### Step 3: Run the tests

```
& "C:\Program Files\Python314\python.exe" -m pytest tests/ --cov=src --cov-report=term-missing
```

### Step 4: (Optional) Re-clean the data

```
from src.data_processing import load_and_clean
load_and_clean("data/healthcare_cleaned.csv", "data/healthcare_for_db.csv")
```

### Step 5: Set up databases

```
# MySQL
CREATE DATABASE healthcare_db; USE healthcare_db;
mysql -u root -p healthcare_db < sql/create_tables_mysql.sql

# PostgreSQL
psql -U postgres -c "CREATE DATABASE healthcare_db;"
psql -U postgres -d healthcare_db -f sql/create_tables.sql
```

### Step 6: Import the data

Use LOAD DATA (MySQL) or \copy (PostgreSQL), or run the Python/SQLAlchemy import shown in db_import.py. The CSV to import is data/healthcare_for_db.csv.

### Step 7: Run the queries

```
psql -U postgres -d healthcare_db -f sql/queries.sql
mysql -u root -p healthcare_db < sql/queries.sql
```

## 13. REQUIREMENTS AND DEPENDENCIES

```
Python 3.14+
pandas
mysql-connector-python
SQLAlchemy
psycopg2-binary
pytest
pytest-cov
fpdf2 (for generating this report)
```

A MySQL server (8.x) and a PostgreSQL server (14+) are needed for the database portions. Both can be installed locally or run via Docker.

## 14. CONCLUSION

This project successfully demonstrates a complete, professional data analysis workflow on a real healthcare dataset:

1. Data quality was assessed and improved: 534 duplicates removed, text standardized, dates and numbers correctly typed. Every fix is documented with affected row counts.

2. The pipeline was modularized into 6 reusable Python functions, making it easy to re-run on new data.

3. The cleaned data was loaded into both MySQL and PostgreSQL using the same table structure, proving database portability.

4. Ten analytical SQL queries answer real questions using GROUP BY, aggregations, and two window functions (RANK for hospital ranking and a running total for monthly billing).

5. The code is validated by 12 pytest tests achieving 100% coverage, well above the 90% target.

6. Concrete findings were extracted: average billing of $25,544.31, six nearly equally common conditions led by Arthritis (9,218 cases), and Obesity being the costliest condition ($25,804.36 average).

The project is reproducible: following the steps in section 12, anyone can recreate the databases, re-run the tests, and reproduce every number in this report. The code and full documentation are available on GitHub at github.com/AryanNagori1405.